

SCS 2312

Computational Models and Programming Language Concepts

Why do we need to study Programming Language Concepts?

Tharindu Wijethilake
tnb@ucsc.cmb.ac.lk



UNIVERSITY OF COLOMBO SCHOOL OF COMPUTING



Tharindu Wijethilake

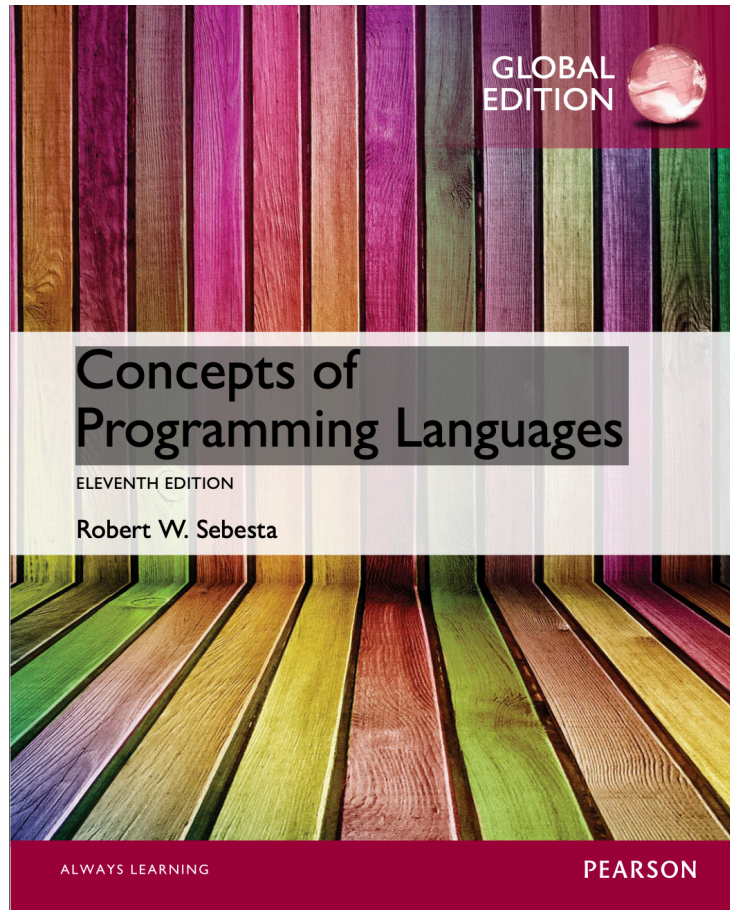
(tnb@ucsc.cmb.ac.lk)

- Undergraduate
 - Computer Networks
 - Computational Models and Programming Language Concepts
 - System and Network Administration
 - Computer Networks II
 - Operating Systems II
- Postgraduate
 - Network Security
 - Security in Mobile and Wireless Security
 - Information and Coding Theory
 - Multimedia Security and Digital Rights Management
 - Data programming
 - Network Design
- Research Interests
 - Computer Networks
 - Network Security
 - Operating Systems

Computational Models and Programming Language Concepts

- Two (2) Hour Paper
- 20% Assignment
- 80% Paper
 - 3 structured questions (75%)
 - 25 MCQ questions with multiple answers and penalties (25%)

Concepts of Programming Languages



Plan for the day.

- Why Study Programming Languages Concepts?
- Evaluate a programming language
- Language Categories
- Translators

Programming Languages

- What is your favourite programming language? Why?



- What is your least favourite programming language? Why?

Programming Languages

Nov 2025	Nov 2024	Change	Programming Language		Ratings	Change
1	1			Python	23.37%	+0.52%
2	4	▲		C	9.68%	+0.67%
3	2	▼		C++	8.95%	-1.69%
4	3	▼		Java	8.54%	-1.06%
5	5			C#	7.65%	+2.67%
6	6			JavaScript	3.42%	-0.29%
7	9	▲		Visual Basic	3.31%	+1.36%

<https://www.tiobe.com/tiobe-index/>

Why Study Programming Languages Concepts?

- Increased capacity to express ideas.
- Improved background for choosing appropriate languages
- Increased ability to learn new languages
- Better understanding of the significance of implementation
- Better use of languages that are already known

Programming domains

- Scientific Applications - Fortran
- Business Applications - COBOL
- Artificial intelligence - Lisp, Prolog, Logic programming
- Systems programming - C
- Web Software

Language evaluation criteria

- Readability

Language evaluation criteria

- Readability

In Python, it's this simple:

```
print "Hello, World!"
```

In Brainfuck, it's:

```
+++++++[[>++++++>+++++++>++++>+<<<<-]>++.>+.+++++. .+++.>+.  
<<+++++++>+. .+++ .----- .----- .>+.> .
```

<https://www.zmescience.com/science/brainfuck-programming/>

Language evaluation criteria

- Readability

Hello, World!

This program displays "Hello, World!".^[10]

```
(=<`#9]~6ZY327Uv4-QsqpMn&+Ij"'E%e{Ab~w=_:]Kw%o44Uqp0/Q?xNvL:`H%c#DD2^WV>gY;dts76qKJImZkj
```

<https://en.wikipedia.org/wiki/Malbolge>

Language evaluation criteria

- Readability
 - Overall simplicity
 - Number of basic constructs
 - Feature complexity
 - Operator overloading
 - Orthogonality
 - Data types
 - Syntax design
 - Special words
 - Form and meaning

Language evaluation criteria

- Writability
 - Simplicity and orthogonality
 - Support of abstraction
 - Process abstraction
 - Data abstraction
- Expressivity

Language evaluation criteria

- Reliability
 - Type checking
 - Exception handling
 - Aliasing
 - Readability and writability

Language evaluation criteria

- Cost
 - Cost of training programmers
 - Cost of writing programmes in the language
 - Cost of compiling programmes written in the language
 - Cost of executing the programs written in the language - Optimisation
 - Cost of the language implementation system
 - Cost of poor reliability
 - Cost of maintaining

Language evaluation criteria

- Portability
- Generality
- Well-definedness - official defining document

Influences on Language Design

- Computer architecture
 - Von Neumann Architecture
 - Imperative languages
 - Functional programming languages
- Programming design methodologies
 - Top-down design
 - Data-oriented program design
 - Object-oriented design

Language categories

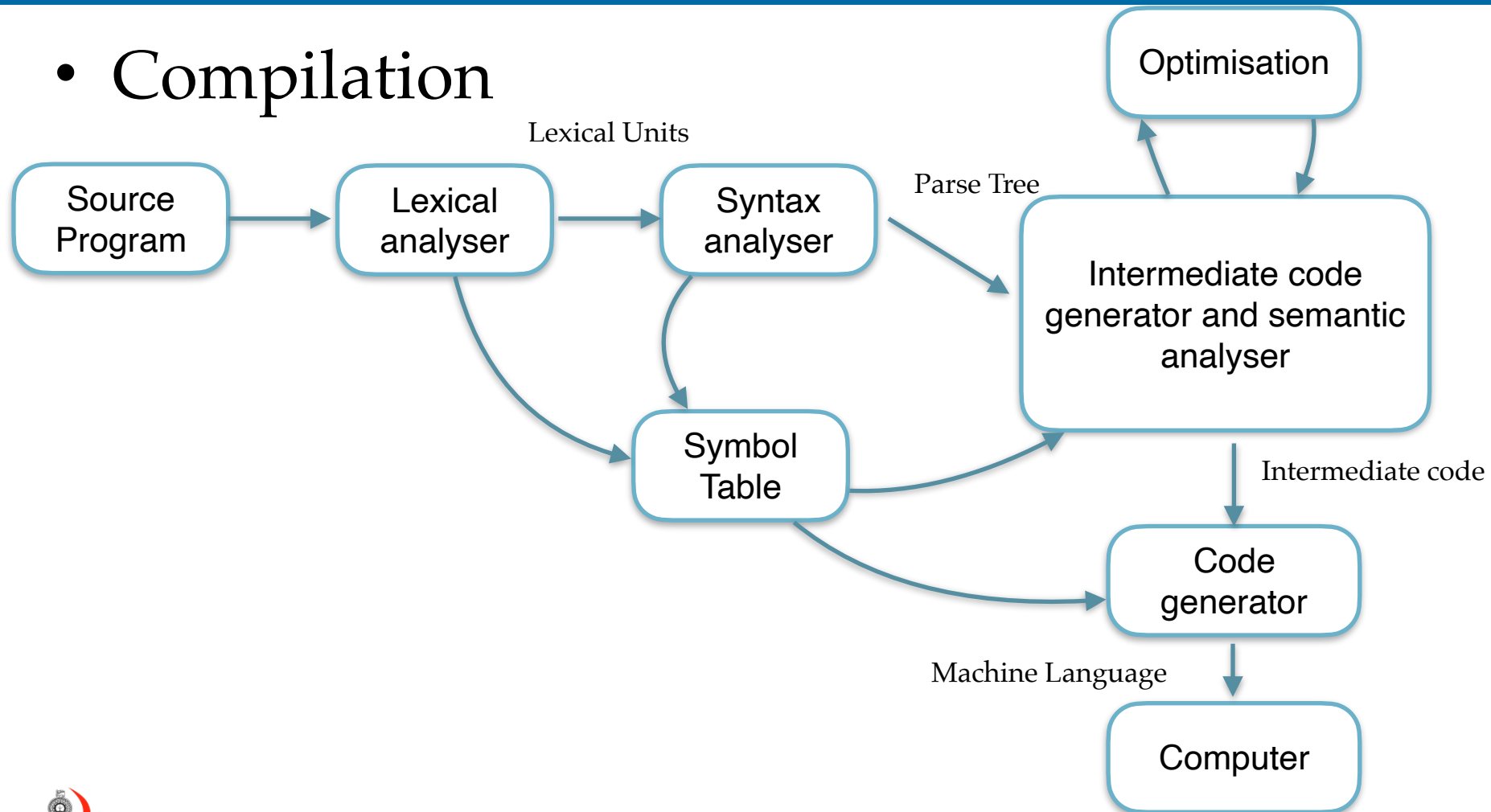
- Imperative
- Functional
- Logic
- Object oriented

Language design tradeoff

- Remember :
 - Readability
 - Writability
 - Reliability
 - Cost
 - Portability
 - Generality ...

Implementation methods

- Compilation

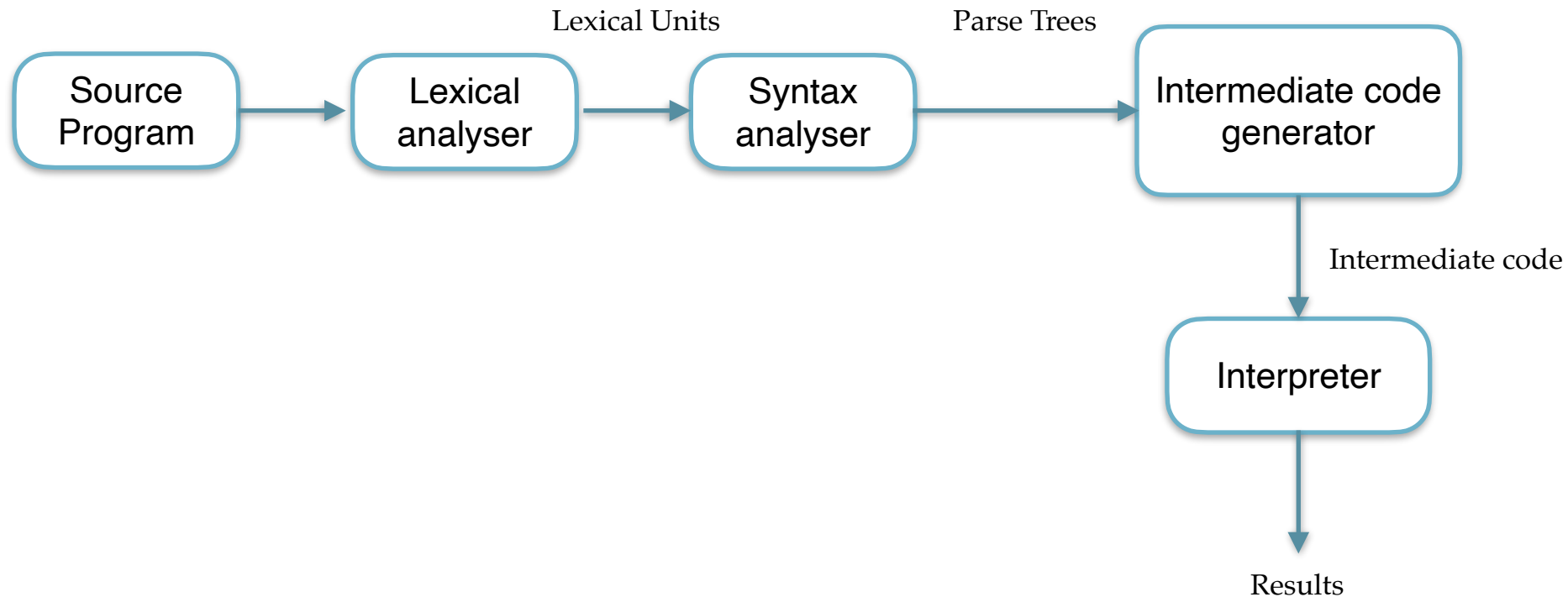


Implementation methods

- Pure Interpretation
 - The program is interpreted by another program called an interpreter, instruction by instruction.

Implementation methods

- Hybrid Implementation Systems



Implementation methods

- Preprocessor
 - Process the source code before the compiler

Programming environments

- A programming environment is the collection of tools used in the development of software.
 - Jbuilder
 - Netbeans
 - Visual Studio